“LINUX PROCESS MONITORING AND CONTROL SYSTEM”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINNERING

By

SAI SURENDRA	**2520030157**
ADA ROHAN	**2520030195**
RAM RATAN	**2520030291**

Under the Esteemed Guidance of

E. SOUMYA
Assistant Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



Declaration

The Project Report entitled **"Linux Process Monitoring And Control System"** is a record of Bonafide work of **SAI SURENDRA - 2520030157 , ADA ROHAN - 2520030195, RAM RATAN - 2520030291** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

SURENDRA 2520030157

ADA ROHAN 2520030195

RAM RATAN 2520030291

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is certify that the mini project based report entitled**"Linux Process Monitoring And Control System"** is a bonafide work done and submitted by **SAI SURENDRA – 2520030157 , ADA ROHAN - 2520030195, RAM RATAN - 2520030291** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator Dr. K. Anusha, and my guide E. Soumya**,** Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Rao (CSE)**, Head of the ***Department for Computer Science Engineering*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

SAI SURENDRA 2520030157

ADA ROHAN 2520030195

RAM RATAN 2520030291

Abstract

Linux Process Monitoring and Control System is a Linux-based systems programming project designed to monitor and control running processes efficiently. The system provides information such as Process ID (PID), process state, CPU and memory usage, and other process-related details. It allows users to search, monitor, and control processes through a command-line interface. The project demonstrates important Operating Systems concepts including process management, process states, process identification, signals, process creation, and termination. Linux system interfaces and POSIX mechanisms are used to obtain process information and perform process control operations. The proposed system provides a simple and interactive environment for understanding how Linux manages processes and resources. The expected outcome is a functional process monitoring and control utility that helps users observe running processes and perform basic process management operations while demonstrating practical Operating Systems and Systems Programming concepts.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		8 - 9
		1.1 Background of the Project	
		1.2 Problem Statement	
		1.3 Scope of the Project	
		1.4 Objectives of the Project	
		1.5 Importance of the Application	
		1.6 Target Users / Audience	
2	OS Concepts and Background Study		10 - 11
		2.1 Operating System Concepts Related to the Project	
		2.2 Existing Methods / Algorithms	
		2.3 Proposed Approach	
		2.4 Advantages of the Proposed Approach	
3	System Requirements		12
		3.1 Hardware Requirements	
		3.2 Software Requirements	
		3.3 Operating System Environment – Ubuntu / WSL	
		3.4 Development Tools and Compilers	
4	Technology and Tools Used		13
		4.1 C / C++ Programming	
		4.2 GCC Compiler	
		4.3 Linux System Calls	
		4.4 POSIX Threads / Synchronization	
		4.5 Linux Commands and Utilities	
		4.6 Other Tools / Libraries Used	
5	System Architecture and Design		14 - 16
		5.1 Overall System Architecture	
		5.2System Workflow	
		5.3 Flowchart / Data Flow	

		5.4 Module Description	
		5.5 Input and Output Design	
6	**Implementation**		**17 - 20**
		6.1 Module-wise Implementation	
		6.2 Algorithm / Pseudocode	
		6.3 Core C / C++ Implementation	
		6.4 Linux System Calls / APIs Used	
		6.5 Process / Thread Implementation	
		6.6 IPC / Synchronization Implementation	
		6.7 Memory Management Implementation	
		6.8 Optional User Interface Implementation	
7	**Execution and Demonstration**		**21 - 22**
		7.1 Compilation in Ubuntu	
		7.2 Execution Commands	
		7.3 Sample Inputs and Outputs	
		7.4 Screenshots of Execution	
		7.5 Demonstration of OS Concepts	
8	**Conclusion**		**23 - 24**
		8.1 Summary of the Project	
		8.2 Objectives Achieved	
		8.3 Results Achieved	
		8.4 Skills and Learning Outcomes	
9	**References**		**25**
		9.1 Books	
		9.2 Research Papers	
		9.3 Linux / GNU Documentation	
		9.4 Online Resources	
10	**Conclusion And Future Scope**		**26**
		10.1 Conclusion	
		10.2 Future Scope	

CHAPTER -1 INTRODUCTION

1.1 Background Of The Project

Linux is a widely used operating system that provides powerful mechanisms for managing processes and system resources. A process is a running instance of a program and has properties such as Process ID (PID), process state, CPU usage, and memory usage. Linux provides system interfaces and POSIX mechanisms to monitor and control these processes. This project focuses on developing a Linux Process Monitoring and Control System that provides users with an interactive way to observe and manage running processes through a command-line interface.

1.2 Problem Statement

Managing and monitoring multiple running processes using traditional Linux commands can be difficult for beginners and users who want a simple interface for process management. Users need to identify processes, view their resource usage, monitor their states, and perform control operations such as terminating processes. Therefore, there is a need for a simple Linux-based system that integrates process monitoring and basic process control operations into a single command-line utility.

1.3 Scope Of The Project

The scope of this project is to develop a command-line-based application for monitoring and controlling running processes in a Linux environment. The system will provide information such as PID, process state, CPU usage, and memory usage. It will allow users to search for specific processes, continuously monitor process information, and perform basic control operations such as sending signals and terminating processes. The project primarily focuses on demonstrating Linux process management and POSIX system programming concepts.

1.4 Objectives Of The Project

The main objective of this project is to develop a Linux-based system that can efficiently monitor and manage currently running processes. The system aims to display important process information such as Process ID (PID), process state, CPU usage, and memory usage through a simple command-line interface. It allows users to search for specific processes and understand process identification and different process states. The project also focuses on implementing process control operations using Linux signals and demonstrating important concepts such as process creation and termination. Overall, the project provides practical knowledge and understanding of Operating Systems, Linux process management, and POSIX system programming concepts.

1.5 Importance Of The Application

The application provides a simple and interactive way to understand how Linux manages processes and system resources. It helps users observe the behavior of running processes and perform basic process management operations. The project is also useful for learning important Operating Systems concepts such as process identification, process states, signals, process creation, and process termination. It connects theoretical concepts with practical Linux and POSIX system programming.

1.6 Target Users / Audience

The target users of this project include students learning Operating Systems and Linux system programming, as well as beginner Linux users who want to understand process management. It is also useful for system programming learners interested in POSIX mechanisms and Linux process-related system interfaces. Developers can use the project to gain practical knowledge of Linux process management, while system administrators can use the basic command-line utility to observe and control running processes.

CHAPTER - 2 OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating Systems Concepts Related To The Project

The project is based on several important Operating System concepts related to process management. A process is a program in execution and is identified by a unique Process ID (PID). Linux maintains different process states such as running, sleeping, stopped, and terminated. The project also involves monitoring CPU and memory usage of processes and understanding how system resources are allocated. Concepts such as process creation, process termination, process identification, process states, signals, and inter-process mechanisms are relevant to the project. Linux system interfaces and POSIX mechanisms are used to obtain process information and perform process control operations.

2.2 Existing Methods / Algorithms

Linux provides several existing commands and system interfaces for monitoring and controlling processes. Commands such as ps, top, and htop can be used to view information about running processes, while commands such as kill and pkill can be used to send signals to processes. The Linux /proc filesystem also provides detailed information about individual processes, including their PID, state, CPU usage, and memory-related information. These existing methods provide the basic functionality required for process monitoring and control, but they are generally available as separate commands and tools.

2.3 Proposed Approach

The proposed system combines important process monitoring and control operations into a single command-line-based application. The system retrieves process information from Linux system interfaces and the /proc filesystem, including PID, process state, CPU usage, and memory usage. Users can search for specific processes, view their details, continuously monitor processes, and perform control operations by sending appropriate Linux signals. The application provides a simple and interactive environment for understanding and managing processes while demonstrating practical Operating System and POSIX programming concepts.

2.4 Advantages Of The Proposed Approach

The proposed approach provides a simple and centralized interface for monitoring and controlling Linux processes. It allows users to view important process information and perform basic process management operations without depending on multiple separate commands. The system helps users understand process identification, process states, resource usage, signals, process creation, and termination in a practical way. It is lightweight, command-line based, and suitable for learning Linux systems programming and Operating System concepts.

CHAPTER - 3 SYSTEM REQUIREMENTS

3.1 Hardware Requirements

The project requires a basic computer or laptop capable of running a Linux environment. A minimum of 4 GB RAM, a dual-core or higher processor, and at least 10 GB of available storage are recommended. A keyboard and monitor are required to interact with the command-line interface.

3.2 Software Requirements

The project requires a basic computer or laptop capable of running a Linux environment. A minimum of 4 GB RAM, a dual-core or higher processor, and at least 10 GB of available storage are recommended. A keyboard and monitor are required to interact with the command-line interface.

3.3 Operating System Environment - Ubuntu / WSL

The application is designed to run in a Linux environment, preferably Ubuntu. Users working on Windows can use WSL (Windows Subsystem for Linux) to create a Linux-based development environment. The Linux environment provides access to process-related system interfaces, the /proc filesystem, signals, system calls, and POSIX mechanisms required for the project.

3.4 Development Tools And Compilers

The project can be developed using a terminal-based editor or an IDE along with the GCC compiler. GCC is used to compile and execute the C/C++ source code. Linux terminal utilities and debugging tools can be used to test process monitoring, process creation, signals, and process termination operations.

CHAPTER - 4 TECHNOLOGY AND TOOLS USED

4.1 C / C++ Programming

C/C++ programming is used to develop the Linux Process Monitoring and Control System. These languages provide access to low-level system programming features and Linux APIs required for process management and resource monitoring.

4.2 GCC Compiler

The GCC (GNU Compiler Collection) is used to compile the C/C++ source code into executable programs. It provides the necessary compilation and linking support for developing applications in the Linux environment.

4.3 Linux System Calls

Linux system calls and system interfaces are used to interact with the operating system. Functions such as fork(), exec(), wait(), kill(), and getpid() can be used for process creation, execution, identification, synchronization, and termination.

4.4 POSIX Threads / Synchronization

POSIX mechanisms can be used to demonstrate process and thread-related concepts and synchronization. Functions and libraries provided by POSIX help manage concurrent operations and coordinate activities when required by the application.

4.5 Linux Commands And Utilities

Linux commands and utilities such as ps, top, htop, kill, pgrep, and grep can be used for process monitoring, searching, and control. These utilities also help in testing and validating the functionality of the proposed system.

4.6 Other Tools / Libraries Used

The project uses standard Linux system libraries, POSIX libraries, and the /proc filesystem to obtain process-related information. A terminal, text editor or IDE, and debugging utilities may also be used during development and testing.

CHAPTER - 5 SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The Linux Process Monitoring and Control System follows a modular architecture in which the user interacts with the system through a monitoring dashboard. The dashboard provides an overview of CPU usage, memory usage, number of processes, and process health. The system collects process information such as Process ID (PID), process name, user, process state, CPU usage, and memory usage from the Linux operating system and processes the collected data for display in the process table. The system also provides features for searching processes using their PID or process name, refreshing process information, and viewing individual process details. A process control module uses Linux system calls and signals to perform basic process management operations such as controlling and terminating processes. The system communicates with the Linux operating system through system calls, POSIX mechanisms, and the /proc filesystem, providing an interactive environment for monitoring system resources and managing running processes.

5.2 System Workflow

The system starts by initializing the process monitoring environment and collecting information from the Linux operating system. The collected data is processed and displayed on the dashboard, including CPU usage, memory usage, total number of processes, and process health information. The process table provides detailed information such as PID, process name, user, status, CPU usage, and memory usage. The user can search for a specific process using its PID or process name, refresh the dashboard to obtain updated information, and select a process to view its details. Based on the user's selection, the system can perform appropriate process control operations using Linux system interfaces and signals. After the operation is completed, the process information is updated and displayed on the dashboard, providing an interactive workflow for monitoring and controlling running processes.

5.3 Flowchart / Data Flow

The system starts by initializing the Linux process monitoring environment and collecting the required system information. It then collects information about the currently running processes and processes and organizes the collected data for further use. The processed information is displayed on the dashboard, providing an overview of system resources and running processes. The user can either search for a specific process by entering its PID or process name or view the list of available processes and select a process to view its details. After selecting or searching for a process, the system displays the relevant process information. The user can then perform process control or signal operations using the available Linux process management mechanisms. Once the operation is completed, the system updates the process information and refreshes the display with the latest data. Finally, the system completes the monitoring and control workflow. The data flow of the proposed system begins when the application is launched. The system identifies the available processes and collects their information from the operating system. CPU and memory utilization are also obtained and processed. The collected information is then displayed on the dashboard and process table. The user can search for a process by entering its PID or process name. The system filters the available process information and displays the matching process. When the user selects a process, the system displays its details and allows supported control operations to be performed. After an operation or refresh, the system collects the latest process information and updates the dashboard.

5.4 Module Description

The Linux Process Monitoring and Control System is divided into several modules to make the application easy to develop, understand, and maintain. The Dashboard Module provides an overall view of the system by displaying CPU usage, memory usage, total number of processes, and process health. The Process Monitoring Module retrieves information about running processes, including PID, process name, user, process state, CPU usage, and memory usage, and updates the information for monitoring. The Process Search Module allows users to search for a specific process using its PID or process name. The Process Details Module displays detailed information about a selected process, helping users understand its current state and resource usage. The Process Control Module provides process management functionality using Linux signals and system calls, allowing supported operations such as sending signals and terminating processes. The System Resource Monitoring Module monitors CPU and memory utilization and presents the information using numerical values and graphical representations. Finally, the Refresh and Update Module periodically collects the latest system and process information and updates the dashboard and process table, enabling users to observe changes in real time.

5.5 Input And Output Design

The input and output design of the system provides a simple and interactive interface for users to monitor and control running processes. The main inputs include the PID or process name entered in the search field, the selection of a process from the process table, and the selection of available process control operations. Users can also use the refresh option to request the latest process information. The system displays the collected information through the dashboard and process table, where the dashboard presents CPU usage, memory usage, total number of processes, and process health. The process table displays details such as PID, process name, user, status, CPU usage, memory usage, and health information. The interface also provides options such as View and Refresh, allowing users to interact with and monitor the process information effectively.

CHAPTER - 6 IMPLEMENTATION

6.1 Module - Wise Implementation

The system is divided into multiple modules for easy implementation and maintenance. The System Initialization Module initializes the application and required resources. The Process Monitoring Module collects information about currently running processes from Linux system interfaces and the /proc filesystem. The Process Search Module allows users to find processes using their PID or process name. The Process Details Module displays detailed information about a selected process. The Process Control Module performs process control operations using Linux signals and system calls. The System Resource Module monitors CPU and memory usage. The Refresh Module updates the process information so that users can observe the latest process status and resource usage.

6.2 Algorithm / Pseudocode

The algorithm starts by initializing the system and collecting information about the running processes. The collected process data is then processed and displayed to the user. The user can search for a process by entering its PID or name or select a process from the displayed list. The system displays the selected process details and allows the user to perform supported process control operations. After the operation, the process information is collected again and the display is refreshed.

Pseudocode:

```
START
Initialize System

WHILE system is running
    Collect System Information
    Collect Process Information
    Process and Organize Data
    Display Process Information

    IF user searches for a process
        Read PID or Process Name
        Display Matching Process
    END IF

    IF user selects a process
        Display Process Details
        Perform Selected Process Control Operation
    END IF

    Update Process Data
    Refresh Display
END WHILE

END
```

6.3 Core C/C++ Implementation

The core implementation uses C/C++ programming to interact with the Linux operating system and obtain process-related information. The program uses standard libraries along with Linux and POSIX APIs to identify processes, retrieve process information, and perform control operations. Functions are organized into separate modules so that process monitoring, searching, displaying information, and process control can be handled independently. The main program coordinates these modules and provides a simple command-line interface for user interaction.

6.4 Linux System Calls / API

The system uses Linux system calls and POSIX APIs to implement process management functionality. The getpid() function is used to obtain the Process ID of the current process, while fork() can be used for process creation. The exec() family of functions can be used to execute another program, and wait() is used for process synchronization. The kill() system call is used to send signals to processes for process control and termination. The /proc filesystem is also used to obtain information about running processes, including process state and resource-related information.

6.5 Process / Thread Implementation

The project primarily focuses on Linux process management. Each running process is identified using its unique Process ID (PID), and information about the process is collected and monitored by the application. Process creation can be demonstrated using the fork() system call, while process execution can be performed using the exec() family of functions. The wait() system call can be used to synchronize parent and child processes. Threads are not the primary focus of the system, but POSIX threads can be used when concurrent monitoring or background processing is required.

6.6 IPC / Synchronization Implementation

Inter-Process Communication and synchronization concepts are considered to demonstrate how processes can coordinate with each other. Linux/POSIX mechanisms such as pipes, signals, and process synchronization functions can be used where required. Signals are particularly important in this project because they allow the monitoring system to communicate control requests to running processes. Functions such as kill() can send signals to a selected process, while wait() can be used to synchronize related processes and prevent improper process termination or execution.

6.7 Memory Management Implementation

Memory management in the project focuses on monitoring the memory usage of running processes and handling dynamically allocated memory safely within the application. Process memory information can be obtained through Linux system interfaces such as the /proc filesystem. The application processes this information and displays the memory usage of individual processes. In the C/C++ implementation, dynamically allocated memory is properly managed using appropriate allocation and deallocation techniques to avoid memory leaks and unnecessary memory consumption.

6.8 Optional User Interface Implementation

The system provides a simple command-line interface for interacting with the process monitoring and control utility. The interface displays process information such as PID, process name, user, process state, CPU usage, and memory usage in an organized format. Users can enter a PID or process name to search for a process, view process details, refresh the displayed information, and perform supported process control operations. The command-line interface keeps the application lightweight while providing an interactive environment for understanding Linux process management.

CHAPTER - 7 EXECUTION AND DEMONSTRATION

7.1 Compilation In Ubuntu

The Linux Process Monitoring and Control System is developed using C/C++ programming and can be compiled in an Ubuntu environment using the GCC compiler. The source code is first saved with the appropriate .c or .cpp extension. The GCC compiler is then used to compile the program and generate an executable file. Any compilation errors are corrected before executing the program. Ubuntu provides the required Linux system interfaces, POSIX libraries, process management functions, and /proc filesystem required for the project.

7.2 Execution Commands

After successful compilation, the generated executable is executed through the Ubuntu terminal. The application starts by collecting information about the running processes and displaying the process monitoring interface. Users can search for processes using their PID or process name, view process details, refresh the process information, and perform supported process control operations. The application can also be executed with appropriate permissions when an operation requires access to processes owned by another user.

7.3 Sample Input And Sample Output

The system accepts inputs such as a Process ID (PID), process name, search option, refresh option, or process control operation. For example, the user can enter a PID to view information about a particular process or enter a process name to search for matching processes. The system then displays information such as PID, process name, user, process state, CPU usage, and memory usage. For a process control operation, the user may select a process and choose an appropriate signal operation. The system then updates the process information and displays the latest status.

7.4 Screenshots Of Execution

The execution of the Linux Process Monitoring and Control System is demonstrated through the Ubuntu terminal and command-line interface. During execution, the system initializes the process monitoring environment and collects information about currently running processes. The application displays details such as Process ID (PID), process name, process state, CPU usage, and memory usage. Users can search for a process using its PID or name, view process details, refresh the displayed information, and perform supported process control operations. The execution demonstrates the practical working of Linux process management, process identification, signals, process states, and system resource monitoring.

7.5 Demonstration Of OS Concepts

The project demonstrates several important Operating Systems concepts through practical implementation. Process identification is demonstrated using Process IDs (PIDs), which uniquely identify running processes. Process states are observed by displaying states such as running, sleeping, stopped, or terminated. Process creation and execution can be demonstrated using Linux functions such as fork() and exec(). Process termination and control are implemented using Linux signals and the kill() system call. The project also demonstrates CPU and memory management by monitoring the resource usage of individual processes. Process synchronization can be demonstrated using mechanisms such as wait(), while the /proc filesystem provides access to information maintained by the Linux kernel. Thus, the execution of the project provides practical understanding of Linux process management and POSIX system programming concepts.

CHAPTER - 8 CONCLUSION

8.1 Summary Of The Project

The Linux Process Monitoring and Control System is a Linux-based systems programming project developed to monitor and control running processes efficiently. The system collects and displays important process information such as PID, process name, process state, CPU usage, and memory usage through a command-line interface. It provides functionality for searching processes, viewing process details, monitoring resource usage, and performing basic process control operations using Linux system interfaces and signals. The project provides a practical implementation of important Operating Systems and systems programming concepts.

8.2 Objectives Achieved

The major objectives of the project were achieved by developing a functional process monitoring and control utility. The system successfully identifies running processes using their PIDs, displays process-related information, allows users to search for specific processes, and provides process control functionality. The project also demonstrates the use of Linux system calls, POSIX mechanisms, signals, process states, process creation, and process termination.

8.3 Results Achieved

The implemented system provides an interactive environment for observing and managing running Linux processes. It successfully collects process information and presents it in an organized format through the command-line interface. Users can search for processes, view their details, monitor CPU and memory usage, and perform supported process control operations. The project therefore demonstrates the practical application of Linux process management and systems programming concepts.

8.4 Skills And Learning Outcomes

Through this project, practical knowledge of Linux and Operating Systems concepts was gained. The project provides experience in C/C++ systems programming, Linux system calls, POSIX APIs, process identification, process states, signals, process creation and termination, synchronization, and resource monitoring. It also improves understanding of how the Linux operating system manages processes and system resources and provides practical experience in developing a command-line systems utility.

CHAPTER - 9 REFERENCES

9.1 Books

- **Operating System Concepts** — Abraham Silberschatz, Peter B. Galvin, Greg Gagne
- **The Linux Programming Interface** — Michael Kerrisk
- **Advanced Programming in the UNIX Environment** — W. Richard Stevens, Stephen A. Rago
- **Modern Operating Systems** — Andrew S. Tanenbaum, Herbert Bos
- **UNIX Network Programming** — W. Richard Stevens

9.2 Research Papers

- **The Linux Process Model** — Linux Kernel Documentation
- **The Linux Scheduler: A Decade of Evolution** — Linux Kernel Scheduling Research
- **Process Scheduling in Linux** — Research literature on Linux scheduling
- **A Study of Linux Process Management** — Operating Systems Research
- **Resource Management in Operating Systems** — Systems Programming Research

9.3 Linux / GNU Documentation

- **Linux Man Pages**
- **GNU C Library (glibc) Documentation**
- **GCC — GNU Compiler Collection Documentation**
- **Linux Kernel Documentation**
- **Linux / proc Filesystem Documentation**
- **POSIX.1 — IEEE Standard for Portable Operating System Interface**
- **Linux fork( ) , exec ( ) , wait ( ) , kill ( ) manual pages**

9.4 Online Resources

- **Linux Documentation Project (LDP)**
- **GeeksforGeeks — Operating Systems**
- **TutorialsPoint — Operating Systems**
- **IBM Developer — Linux and Systems Programming**

CHAPTER - 10 CONCLUSION AND FUTURE SCOPE

10.1 Conclusion

The Linux Process Monitoring and Control System successfully demonstrates the practical implementation of Linux process management and Operating Systems concepts. The system provides a simple way to monitor running processes by displaying information such as Process ID (PID), process state, CPU usage, and memory usage. It also demonstrates process identification, process creation and termination, Linux signals, system calls, and POSIX mechanisms. Overall, the project provides practical knowledge of Linux systems programming and helps in understanding how the operating system manages processes and system resources.

10.2 Future Scope

The project can be further enhanced by adding advanced monitoring and process management features. Future versions can include a graphical user interface, real-time CPU and memory usage graphs, automatic process alerts, process priority management, detailed process history, and improved filtering and sorting options. Additional features such as process tree visualization, logging of process activities, remote process monitoring, and support for multiple Linux systems can also be implemented. These enhancements can make the system more powerful and suitable for practical system monitoring and administration.